

Manual De Instalación Y Primer Desarrollo

Curso: Realidad Virtual y Aumentada

Propósito: instalar, verificar y dejar operativo un entorno base para proyectos de RA, RV, RM y XR en equipo.

Tecnologías incluidas: Unity, AR Foundation, ARCore, Vuforia, OpenXR, XR Interaction Toolkit, GitHub.

1. Introducción

Este manual sirve para que cada grupo pueda:

- instalar el entorno base desde cero;
- verificar que su equipo funciona correctamente;
- crear un proyecto base común llamado XR_Starter_Lab;
- construir 4 escenas mínimas de referencia;
- decidir con criterio qué stack usar para su proyecto final.

Después de seguir este manual podrán desarrollar, según el proyecto:

- RA sobre superficies: Unity + AR Foundation + ARCore
- RA con imágenes / marcadores: Unity + Vuforia
- RV con visor: Unity + OpenXR + XR Interaction Toolkit
- demo 3D base: Unity

Advertencia

No instalen todo “por si acaso”. Primero definan el tipo de experiencia que van a construir.

2. Requisitos Previos

Hardware mínimo

- CPU Intel i5 / Ryzen 5 o superior
- 8 GB RAM mínimo
- 20 a 30 GB libres en disco
- GPU integrada moderna o dedicada básica
- conexión a internet estable

Hardware recomendado

- 16 GB RAM
- SSD
- GPU dedicada
- celular Android compatible con ARCore
- visor XR solo si el proyecto realmente lo requiere

Sistema operativo

- Windows 10/11
- macOS reciente compatible con Unity Hub y Unity Editor

Cuentas necesarias

- cuenta Unity

- cuenta Vuforia
- cuenta GitHub

Dispositivos

- ARCore: requiere celular Android compatible
- Vuforia: puede probarse con webcam o móvil, pero el objetivo real es móvil
- OpenXR: idealmente visor compatible; si no hay visor, usar escena conceptual de RV simple

Verificación

Antes de continuar, cada grupo debe confirmar:

- un equipo principal de desarrollo;
- un repositorio común;
- al menos un celular Android si hará RA móvil;
- quién será responsable de instalación.

3. Tabla De Decisión Tecnológica

Tipo de proyecto	Stack recomendado	Cuándo usarlo	Cuándo no usarlo
RA sobre mesa, piso o superficie real	Unity + AR Foundation + ARCore	cuando el usuario coloca objetos en el entorno	si todo depende de una imagen target
RA con imagen, póster, lámina o marcador	Unity + Vuforia	cuando se necesita reconocimiento visual específico	si solo se quiere detectar superficies
RV con visor	Unity + OpenXR + XR Interaction Toolkit	cuando hay entorno inmersivo completo	si no tienen visor y el proyecto depende de él
Demo 3D base o prototipo inicial	Unity	para aprender escenas, scripts y lógica	si ya necesitan tracking AR o soporte XR real
WebXR simple	A-Frame / WebXR opcional	para demos web ligeras	si el proyecto necesita capacidades fuertes de Unity

Recomendación

Para grupos principiantes, la ruta más segura suele ser:

- AR Foundation + ARCore o
- Vuforia

4. Instalación De Unity Hub

Recurso oficial

- Unity Hub: <https://docs.unity.com/en-us/hub>
- Instalar Unity Hub: <https://docs.unity.com/en-us/hub/install-hub>

Pasos

1. Entren a Unity Hub.
2. Descarguen la versión para su sistema operativo.
3. Instalen la aplicación.
4. Inicien sesión con su cuenta Unity.

Qué hace Unity Hub

- instala editores de Unity;
- gestiona proyectos;
- agrega módulos como Android Build Support.

Errores comunes

- no iniciar sesión;
- instalar Hub pero no el Editor;
- no tener permisos de instalación.

Verificación

- Unity Hub abre correctamente.
 - Pueden ver las pestañas Projects e Installs.
-

5. Instalación Del Editor De Unity

Recursos oficiales

- Android environment setup: <https://docs.unity.cn/Documentation/Manual/android-sdksetup.html>
- AR Foundation manual: <https://docs.unity.cn/Manual/com.unity.xr.arfoundation.html>

Versión recomendada

Use Unity 2022.3 LTS.

Por qué

- es estable;
- tiene mejor compatibilidad con paquetes del curso;
- es adecuada para laboratorio.

Pasos

1. Abrir Unity Hub.
2. Ir a Installs.
3. Elegir Install Editor.
4. Seleccionar Unity 2022.3 LTS.
5. Marcar estos módulos:
 - Android Build Support
 - Android SDK & NDK Tools
 - OpenJDK

Opcionales

- Windows Build Support si usarán PC/VR
- Documentation no es obligatoria

Qué debe quedar instalado

- Unity Editor
- SDK Android
- NDK Android
- JDK

Advertencia

Si olvidan Android Build Support, luego ARCore no compilará correctamente a Android.

Verificación

- El editor aparece en Installs.
 - El editor abre sin errores.
 - Los módulos de Android figuran instalados.
-

6. Creación Del Proyecto Base

Pasos

1. En Unity Hub, clic en New Project.
2. Elegir plantilla 3D Core.
3. Nombre: XR_Starter_Lab.
4. Crear el proyecto.

Estructura recomendada

Dentro de Assets, creen estas carpetas:

- Scenes
- Scripts
- Prefabs
- Materials
- Docs
- Videos
- Capturas

Buenas prácticas

- una escena por tecnología;
- scripts con nombres claros;
- no mezclar archivos sin orden.

Verificación

El proyecto abre y la carpeta Assets ya está organizada.

7. Instalación De AR Foundation Y ARCore

Recursos oficiales

- AR Foundation manual: <https://docs.unity.cn/Manual/com.unity.xr.arfoundation.html>
- ARCore overview: <https://developers.google.com/ar/develop>
- ARCore con Unity/AR Foundation: <https://developers.google.com/ar/develop/unity-arf/getting-started>
- AR Foundation Samples: <https://github.com/Unity-Technologies/arfoundation-samples>
- AR Foundation Demos: <https://github.com/Unity-Technologies/arfoundation-demos>

Qué es cada uno

- AR Foundation: capa de Unity para trabajar con RA.
- ARCore: tecnología de Google para RA en Android.

Diferencia

- AR Foundation integra.
- ARCore aporta capacidades de RA del dispositivo Android.

Instalación

1. Ir a Window > Package Manager.
2. En Unity Registry, buscar AR Foundation.
3. Instalar.
4. Buscar ARCore XR Plugin.
5. Instalar.

Configuración Android

1. Ir a File > Build Settings.
2. Seleccionar Android.
3. Clic en Switch Platform.

Compatibilidad

Para saber si un celular soporta ARCore:

- <https://developers.google.com/ar/devices>

Advertencia

No todos los Android soportan ARCore.

Verificación

- El proyecto cambió a Android.
 - AR Foundation y ARCore XR Plugin aparecen instalados en Package Manager.
-

8. Instalación De Vuforia

Recursos oficiales

- Getting Started with Vuforia in Unity: <https://developer.vuforia.com/library/getting-started/getting-started-vuforia-engine-unity?partnerID=Unity>
- Image Targets: <https://developer.vuforia.com/library/objects/image-targets>
- Vuforia Engine API / lifecycle: <https://developer.vuforia.com/library/vuforia-engine/getting-started/vuforia-engine-api/vuforia-engine-api-unity/>

Cuándo usarlo

- cuando el proyecto depende de una imagen;
- cuando el contenido aparece al enfocar una tarjeta, lámina, afiche o panel.

Pasos

1. Crear cuenta en Vuforia Developer Portal.
2. Obtener una License Key.
3. Descargar el paquete/extensión de Vuforia para Unity.
4. En Unity:
 - Assets > Import Package > Custom Package
5. Importar el paquete.

6. Agregar ARCamera.
7. Abrir Vuforia Configuration.
8. Pegar la licencia.

Image Target

1. Crear o cargar una imagen target.
2. Agregar Image Target en la escena.
3. Colocar el objeto 3D como hijo del target.

Advertencia

Si la imagen target tiene pocas características visuales, el tracking será malo.

Verificación

- El menú de Vuforia aparece en Unity.
 - La licencia fue aceptada.
 - El Image Target existe en escena.
-

9. Instalación De OpenXR Y XR Interaction Toolkit

Recursos oficiales

- OpenXR Plugin Unity: <https://docs.unity.cn/Manual/com.unity.xr.openxr.html>
- XR Interaction Toolkit manual: <https://docs.unity.cn/2022.3/Documentation/Manual/com.unity.xr.interaction.toolkit.html>
- XR Interaction Toolkit Examples: <https://github.com/Unity-Technologies/XR-Interaction-Toolkit-Examples>
- OpenXR registry: <https://registry.khronos.org/OpenXR/>

Qué es OpenXR

Estándar abierto para trabajar con múltiples dispositivos XR.

Qué es XR Interaction Toolkit

Framework de interacción para RA/RV/XR en Unity.

Instalación

1. Window > Package Manager
2. Instalar XR Plug-in Management
3. Instalar OpenXR Plugin
4. Instalar XR Interaction Toolkit

Activación

1. Ir a Edit > Project Settings > XR Plug-in Management
2. Activar OpenXR

Si no hay visor

Igual pueden:

- instalar y configurar;
- crear una escena VR conceptual;
- ejecutar scripts de detección;
- dejar el stack listo para futura prueba.

Verificación

- OpenXR aparece activado.

- XR Interaction Toolkit aparece instalado.

10. Herramientas Adicionales Recomendadas

Herramienta	¿Obligatoria?	Uso
GitHub Desktop	Opcional recomendada	simplifica Git para principiantes
Visual Studio Code o Visual Studio	Sí	editar scripts C#
OBS Studio	Opcional recomendada	grabar demos y tutoriales
Figma	Opcional	storyboard y UX
Blender	Opcional	modelado 3D básico

Recursos

- GitHub Desktop: <https://desktop.github.com/>
 - VS Code: <https://code.visualstudio.com/>
 - OBS Studio: <https://obsproject.com/>
 - Figma: <https://www.figma.com/>
 - Blender: <https://www.blender.org/>
-

11. Configuración Del Repositorio Del Proyecto

Por qué usar GitHub

- trabajo colaborativo;
- respaldo;
- control de versiones;
- evidencia del desarrollo semanal.

Pasos

1. Crear repositorio en GitHub.
2. Nombre sugerido: grupoX-xr-starter-lab
3. Subir el proyecto base.

Estructura recomendada

```
XR_Starter_Lab/
```

```
|— Assets/
```

```
|— Packages/
```

```
|— ProjectSettings/
```

```
|— Docs/
```

```
|— Videos/  
|— Capturas/  
|— README.md
```

README inicial

Debe incluir:

- nombre del proyecto
- integrantes
- problema
- stack tecnológico
- cómo abrir el proyecto
- estado actual

Qué sí subir

- Assets
- Packages
- ProjectSettings
- Docs
- README

Qué no subir

- carpetas temporales del sistema
- archivos innecesarios del editor no rastreados si usan `.gitignore` de Unity

Recurso

- GitHub: <https://github.com/>
- Unity `.gitignore` template: <https://github.com/github/gitignore/blob/main/Unity.gitignore>

12. Desarrollo Guiado De 4 Escenas Mínimas

Escena 1: Unity Básico, Un Cubo Que Gira

Objetivo

Comprobar que Unity funciona, que los estudiantes saben crear escena y agregar un script.

Pasos

1. Crear escena 01_Unity_Basico.
2. Guardarla en Assets/Scenes.
3. Agregar:
 - GameObject > 3D Object > Plane
 - GameObject > 3D Object > Cube
4. Colocar el cubo en posición (0,1,0).
5. Crear script SpinObject.cs en Assets/Scripts.
6. Asignar el script al cubo.

Código

```
using UnityEngine;

public class SpinObject : MonoBehaviour
{
    public float speed = 60f;

    void Update()
    {
        transform.Rotate(Vector3.up * speed * Time.deltaTime);
    }
}
```

Explicación

- `using UnityEngine;`: importa el núcleo de Unity.
- `MonoBehaviour`: permite que el script se agregue a un objeto.
- `speed`: velocidad editable desde el Inspector.
- `Update()`: se ejecuta cada frame.
- `transform.Rotate(...)`: rota el cubo.
- `Time.deltaTime`: hace el movimiento consistente.

Qué se debe observar

Al presionar Play, el cubo gira.

Errores comunes

- olvidar guardar el script;
- nombre del archivo distinto al nombre de la clase;
- no arrastrar el script al cubo.

Verificación

- El cubo gira al ejecutar.

Escena 2: ARCore / AR Foundation, Colocar Un Cubo Sobre Una Mesa

Objetivo

Detectar una superficie real y colocar un cubo tocando la pantalla.

Pasos

1. Crear escena 02_ARCore_TapToPlace.

2. Eliminar la cámara principal si hace falta.
3. Agregar:
 - AR Session
 - XR Origin (Mobile AR) o equivalente según versión
4. En el objeto base de AR:
 - agregar AR Plane Manager
 - agregar AR Raycast Manager
5. Crear un cubo y convertirlo en prefab.

Script

```
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.XR.ARFoundation;
using UnityEngine.XR.ARSubsystems;

public class TapToPlaceObject : MonoBehaviour
{
    public GameObject prefabToPlace;
    private ARRaycastManager raycastManager;
    private GameObject spawnedObject;
    private static List<ARRaycastHit> hits = new List<ARRaycastHit>();

    void Awake()
    {
        raycastManager = GetComponent<ARRaycastManager>();
    }

    void Update()
    {
        if (Input.touchCount == 0) return;

        Touch touch = Input.GetTouch(0);
        if (touch.phase != TouchPhase.Began) return;
```

```
        if (raycastManager.Raycast(touch.position, hits,
TrackableType.PlaneWithinPolygon))
        {
            Pose hitPose = hits[0].pose;

            if (spawnedObject == null)
                spawnedObject = Instantiate(prefabToPlace, hitPose.position,
hitPose.rotation);
            else

spawnedObject.transform.SetPositionAndRotation(hitPose.position,
hitPose.rotation);
        }
    }
}
```

Explicación

- ARRaycastManager: lanza rayos sobre superficies detectadas.
- Input.touchCount: verifica toque en pantalla.
- TrackableType.PlaneWithinPolygon: usa planos detectados.
- Pose: posición y rotación donde se colocará el objeto.

Compilación

1. File > Build Settings
2. Android activo
3. Conectar celular
4. Build and Run

Qué se debe observar

Al tocar una mesa o piso detectado, aparece el cubo.

Errores comunes

- celular no compatible con ARCore;
- no cambiar plataforma a Android;
- no agregar AR Session;
- prefab sin asignar.

Verificación

- La app abre en el celular.
 - Detecta superficie.
 - El cubo aparece con toque.
-

Escena 3: Vuforia, Mostrar Un Cubo Sobre Una Imagen Target

Objetivo

Activar un objeto 3D al reconocer una imagen.

Pasos

1. Crear escena 03_Vuforia_ImageTarget.
2. Agregar ARCamera.
3. Pegar la licencia en Vuforia Configuration.
4. Crear Image Target.
5. Poner un cubo como hijo del Image Target.
6. Escalar el cubo para que se vea correctamente.

Script opcional

```
using UnityEngine;
using Vuforia;

public class VuforiaTargetEvents : MonoBehaviour
{
    private ObserverBehaviour observerBehaviour;

    void Start()
    {
        observerBehaviour = GetComponent<ObserverBehaviour>();

        if (observerBehaviour != null)
            observerBehaviour.OnTargetStatusChanged += OnTargetStatusChanged;
    }

    void OnDestroy()
    {
        if (observerBehaviour != null)
            observerBehaviour.OnTargetStatusChanged -= OnTargetStatusChanged;
    }
}
```

```
private void OnTargetStatusChanged(ObserverBehaviour behaviour,
TargetStatus status)
{
    if (status.Status == Status.TRACKED || status.Status ==
Status.EXTENDED_TRACKED || status.Status == Status.DETECTED)
        Debug.Log("Target encontrado");
    else
        Debug.Log("Target perdido");
}
```

Qué hace

Muestra en consola si el target fue encontrado o perdido.

Prueba

- imprimir la imagen target o mostrarla en otra pantalla;
- iniciar la app;
- apuntar la cámara al target.

Qué se debe observar

Cuando la imagen es detectada, aparece el cubo.

Errores comunes

- licencia inválida;
- imagen target mal configurada;
- target con pocas características visuales;
- cubo no es hijo del target.

Verificación

- Al enfocar la imagen, aparece el cubo.
- La consola registra detección.

Escena 4: OpenXR, Detectar Dispositivo XR O Mostrar Escena VR Simple

Objetivo

Comprobar que OpenXR está activo y dejar preparada la base para VR.

Pasos

1. Crear escena 04_OpenXR_Basico.
2. Confirmar que OpenXR está activado.
3. Crear una escena 3D simple con piso y objetos.
4. Agregar una cámara base o rig XR según el paquete instalado.

Script

```
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.XR;

public class XRDeviceProbe : MonoBehaviour
{
    void Start()
    {
        List<InputDevice> devices = new List<InputDevice>();
        InputDevices.GetDevices(devices);

        foreach (var device in devices)
        {
            Debug.Log("Dispositivo XR detectado: " + device.name + " | " +
device.characteristics);
        }
    }
}
```

Qué pasa si no hay visor

- la escena igual puede existir;
- el stack queda preparado;
- el script puede no detectar hardware, pero la configuración queda documentada.

Qué se debe observar

Si existe un dispositivo XR, se verá en la consola.

Errores comunes

- no activar OpenXR en Project Settings;
- esperar inmersión real sin visor;

- no instalar XR Interaction Toolkit cuando el proyecto sí lo necesita.

Verificación

- OpenXR activo.
 - Escena creada.
 - Script ejecuta sin errores.
-

13. Código Completo

SpinObject.cs

```
using UnityEngine;

public class SpinObject : MonoBehaviour
{
    public float speed = 60f;

    void Update()
    {
        transform.Rotate(Vector3.up * speed * Time.deltaTime);
    }
}
```

TapToPlaceObject.cs

```
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.XR.ARFoundation;
using UnityEngine.XR.ARSubsystems;

public class TapToPlaceObject : MonoBehaviour
{
    public GameObject prefabToPlace;
    private ARRaycastManager raycastManager;
    private GameObject spawnedObject;
    private static List<ARRaycastHit> hits = new List<ARRaycastHit>();
```

```
void Awake()
{
    raycastManager = GetComponent<ARRaycastManager>();
}

void Update()
{
    if (Input.touchCount == 0) return;

    Touch touch = Input.GetTouch(0);
    if (touch.phase != TouchPhase.Began) return;

    if (raycastManager.Raycast(touch.position, hits,
TrackableType.PlaneWithinPolygon))
    {
        Pose hitPose = hits[0].pose;

        if (spawnedObject == null)
            spawnedObject = Instantiate(prefabToPlace, hitPose.position,
hitPose.rotation);
        else

spawnedObject.transform.SetPositionAndRotation(hitPose.position,
hitPose.rotation);
    }
}
```

VuforiaTargetEvents.cs

```
using UnityEngine;

using Vuforia;

public class VuforiaTargetEvents : MonoBehaviour
```



```
{  
    private ObserverBehaviour observerBehaviour;  
  
    void Start()  
    {  
        observerBehaviour = GetComponent<ObserverBehaviour>();  
  
        if (observerBehaviour != null)  
            observerBehaviour.OnTargetStatusChanged += OnTargetStatusChanged;  
    }  
  
    void OnDestroy()  
    {  
        if (observerBehaviour != null)  
            observerBehaviour.OnTargetStatusChanged -= OnTargetStatusChanged;  
    }  
  
    private void OnTargetStatusChanged(ObserverBehaviour behaviour,  
    TargetStatus status)  
    {  
        if (status.Status == Status.TRACKED || status.Status ==  
        Status.EXTENDED_TRACKED || status.Status == Status.DETECTED)  
            Debug.Log("Target encontrado");  
        else  
            Debug.Log("Target perdido");  
    }  
}
```

XRDeviceProbe.cs

```
using System.Collections.Generic;  
using UnityEngine;  
using UnityEngine.XR;
```

```
public class XRDeviceProbe : MonoBehaviour
{
    void Start()
    {
        List<InputDevice> devices = new List<InputDevice>();
        InputDevices.GetDevices(devices);

        foreach (var device in devices)
        {
            Debug.Log("Dispositivo XR detectado: " + device.name + " | " +
device.characteristics);
        }
    }
}
```

14. Qué Proyecto Puede Tomar Cada Grupo Según La Herramienta

Proyecto	Tecnología recomendada	Dificultad
Sistema Solar en RA	Unity + Vuforia	Baja
Tutor de Anatomía en RA	AR Foundation o Vuforia	Baja-media
AR PC Hardware Tutor	Vuforia o AR Foundation	Baja-media
Digital Twin de Laboratorio	Unity base o Unity + OpenXR	Media
AR Museum / Heritage Guide	Vuforia	Baja
AR First Aid Assistant	Vuforia	Baja
Indoor Navigation	AR Foundation + ARCore	Media
Escape Room VR	OpenXR + XRI	Media-alta
VR Museum	OpenXR + XRI	Media

Proyecto	Tecnología recomendada	Dificultad
Maintenance Assistant	Vuforia o AR Foundation	Media

Recomendación

- Si el proyecto depende de una imagen: Vuforia.
- Si depende del entorno real y superficies: AR Foundation + ARCore.
- Si depende de inmersión y visor: OpenXR + XRI.

15. Recursos Oficiales Y GitHub

Oficiales

- Unity Hub: <https://docs.unity.com/en-us/hub>
- Instalar Unity Hub: <https://docs.unity.com/en-us/hub/install-hub>
- Unity Manual: <https://docs.unity3d.com/Manual/index.html>
- Android setup: <https://docs.unity.cn/Documentation/Manual/android-sdksetup.html>
- AR Foundation manual: <https://docs.unity.cn/Manual/com.unity.xr.arfoundation.html>
- ARCore overview: <https://developers.google.com/ar/develop>
- ARCore Unity setup: <https://developers.google.com/ar/develop/unity-arf/getting-started>
- ARCore supported devices: <https://developers.google.com/ar/devices>
- Vuforia Unity getting started: <https://developer.vuforia.com/library/getting-started/getting-started-vuforia-engine-unity?partnerID=Unity>
- Vuforia Image Targets: <https://developer.vuforia.com/library/objects/image-targets>
- OpenXR in Unity: <https://docs.unity.cn/Manual/com.unity.xr.openxr.html>
- XR Interaction Toolkit: <https://docs.unity.cn/2022.3/Documentation/Manual/com.unity.xr.interaction.toolkit.html>
- OpenXR registry: <https://registry.khronos.org/OpenXR/>

GitHub

- AR Foundation Samples: <https://github.com/Unity-Technologies/arfoundation-samples>
- AR Foundation Demos: <https://github.com/Unity-Technologies/arfoundation-demos>
- XR Interaction Toolkit Examples: <https://github.com/Unity-Technologies/XR-Interaction-Toolkit-Examples>
- Unity .gitignore: <https://github.com/github/gitignore/blob/main/Unity.gitignore>

16. Errores Frecuentes Y Cómo Solucionarlos

Unity no abre

Causa probable

- instalación incompleta;
- editor corrupto;
- falta de permisos.

Solución

- reinstalar desde Unity Hub;
- ejecutar como administrador si aplica;
- verificar espacio en disco.

Falta Android SDK/NDK

Causa

- módulos no marcados al instalar Unity.

Solución

- agregar módulos desde Unity Hub > Installs > Add modules.

ARCore no detecta planos

Causa

- celular no compatible;
- escena mal configurada;
- poca textura visual en el entorno.

Solución

- verificar compatibilidad;
- revisar AR Session, AR Plane Manager, AR Raycast Manager;
- probar en superficie bien iluminada.

Vuforia no reconoce target

Causa

- imagen pobre;
- licencia mal configurada;
- target mal cargado.

Solución

- usar imagen con más características;
- revisar licencia;
- reconfigurar target.

OpenXR no detecta visor

Causa

- no hay dispositivo;
- runtime no activo;
- plugin no habilitado.

Solución

- revisar XR Plug-in Management;
- verificar visor y runtime;
- documentar que la escena queda conceptual si no hay hardware.

Package Manager no muestra paquetes

Causa

- filtro incorrecto o problema de red.

Solución

- cambiar a Unity Registry;
- reiniciar editor;
- verificar conexión.

Error al compilar Android

Causa

- SDK/NDK/JDK faltantes;
- plataforma no cambiada.

Solución

- revisar módulos;
- hacer Switch Platform a Android.

Namespaces faltantes**Causa**

- script con using incorrecto o paquete no instalado.

Solución

- revisar imports;
 - confirmar instalación del paquete.
-

17. Buenas Prácticas Para Trabajar En Grupo

- Definir roles:
 - líder técnico
 - integrador
 - diseñador UX
 - documentación
 - testing
- Hacer commits semanales.
- Mantener carpeta Docs.
- Grabar avances breves cada semana.
- No mezclar todos los experimentos en una sola escena.
- Tener una demo mínima primero.
- Documentar:
 - qué se hizo,
 - qué falló,
 - cómo se resolvió.

Preparación de hitos

- Semana 4: problema, stack, entorno, MVP.
 - Semana 8: flujo principal funcional.
 - Semana 16: solución completa, estable y documentada.
-

18. Checklist De Instalación Por Equipo

- Unity Hub instalado
- Cuenta Unity activa
- Unity Editor 2022.3 LTS instalado
- Android Build Support instalado

- Android SDK & NDK Tools instalados
 - OpenJDK instalado
 - Proyecto XR_Starter_Lab creado
 - Carpetas del proyecto organizadas
 - Repositorio GitHub creado
 - README inicial creado
 - AR Foundation instalado
 - ARCore XR Plugin instalado
 - Proyecto cambiado a Android
 - Vuforia instalado
 - License Key de Vuforia configurada
 - OpenXR instalado
 - XR Interaction Toolkit instalado
 - OpenXR activado en Project Settings
 - Escena 1 funcionando
 - Escena 2 funcionando
 - Escena 3 funcionando
 - Escena 4 funcionando
 - Evidencias guardadas en Docs/Videos/Capturas
-

19. Cierre

No intenten usar todo al mismo tiempo. Lo correcto es:

1. definir el tipo de proyecto;
2. elegir el stack adecuado;
3. construir una demo mínima funcional;
4. validar que el entorno funciona;
5. iterar semana a semana.

Recomendación final

Si su grupo recién empieza:

- primero logren Escena 1;
- luego elijan entre Escena 2 o Escena 3;
- dejen Escena 4 para los grupos que realmente trabajarán con RV/XR.